

Buildings Built in Minutes - SfM

RBE 549: P2

Pau Alcolea
Department of Robotics
Worcester Polytechnic Institute
palcolea@wpi.edu

Ben Orr
Department of Robotics
Worcester Polytechnic Institute
bcorr@wpi.edu

Abstract—3-Dimensional construction of objects and settings is a subject with a lot of demand in our modern day, specially in the field of robotics, where localization and scanning is used in abundance. The first part of this project, focused on Structure from Motion (SfM), a method of getting 3D models from a series of 2D images, using feature correspondences, estimates of the camera poses and various layers of adjustments with non-linear optimization methods for the reprojection and a final step of bundle adjustments to finalize the pipeline. The second part of this project is an implementation of NeRF, which is an approach to novel view synthesis via a fully-connected network that approximating a continuous volumetric scene function.

I. PROBLEM STATEMENT

In Phase 1 of this project we learn how to reconstruct a 3D scene and simultaneously obtain the camera poses of a monocular camera with respect to the given scene. We are using a series of 2D images of the same scene with overlapping features from slightly different angles, attempting to reconstruct the scene with a single camera in motion.

In Phase 2, we demonstrate a method of synthesizing novel views of complex scenes by optimizing an underlying continuous volumetric scene function using a sparse set of input views.

II. PHASE 1: CLASSICAL APPROACH TO THE SFM PROBLEM

The classical approach to 3D modeling from a set of 2D images from slightly different angles begins with the set of corresponding points between images. These point correspondences are based on features, which are used to estimate the camera poses at each of the frames. Using the Fundamental and the Essential matrix, the 3D World Points are obtained, creating the map of the space one image at a time. This map is cleaned up through RANSAC to remove outliers, and the geometric error (looking at the reprojection of the found world points back into the images), refines the camera poses and the world point coordinates in a series of triangulation and PnP. The very end of the classical pipeline uses Bundle Adjustment to refine all the camera poses and 3D points at once in a final step.

A. Feature Matching, Fundamental Matrix and RANSAC

Feature matching is crucial for SfM, as it finds the corresponding points between two images and establishes a

geometric constraint that allows us to estimate poses and other information. In this project, the feature matching was done ahead of time and was stored in matching.txt files. Each file corresponded to an image and contained all of the points in other images that had a point in common with the picture in question. SIFT was used, ensuring that the matches are robust, crucial for the pipeline, as everything relies on good corresponding points between images.

The images in question were taken with the same camera, which was calibrated using using a Radial-Tangential model in MATLAB R2022a's Camera Calibrator Application. The intrinsic camera matrix contained this information, and was also given to as part of the data for this project.

The fundamental matrix F was estimated using a standard RANSAC implementation, as apparent in Algorithm 1. To increase the accuracy of our F estimates, we adopted the Sampson distance as our error metric, as is used in MATLAB's `estimateFundamentalMatrix()` [1], [2]. While the standard algebraic error can be adequate for testing the epipolar constraint (Equation (1)), it has arbitrary scale and therefore doesn't encode geometric relationships. The Sampson distance squares the algebraic error and divides it by its gradient magnitude, returning a normalized difference in squared pixels, i.e. the first order approximation of geometric distance (Equation (2)).

$$x'^T F x = 0 \quad (1)$$

$$d_S(\mathbf{x}, \mathbf{x}', F) = \frac{(\mathbf{x}'^T F \mathbf{x})^2}{(F \mathbf{x})_1^2 + (F \mathbf{x})_2^2 + (F^T \mathbf{x}')_1^2 + (F^T \mathbf{x}')_2^2} \quad (2)$$

B. Estimating Fundamental Matrix

To estimate the fundamental matrix F requires a minimum of 8 non-collinear correspondences. Between two images, 8 pairs of points (like eq. (3)) will give us 8 degrees of freedom, which will be enough to solve for our 3x3 matrix F because one of the degrees of freedom is taken up by the scale.

The fundamental matrix encodes the epipolar geometry between two images, and it tells that x' must lie on the epipolar line Fx . That epipolar line on the second image corresponds to the ray that belongs to the point x on the first. This leads us



Fig. 1. Dataset provided for Phase 1

Algorithm 1 RANSAC for Fundamental Matrix Estimation

Require: Correspondences $\mathcal{C} = \{(\mathbf{x}_i, \mathbf{x}'_i)\}_{i=1}^N$

Ensure: Fundamental Matrix F

```

1:  $M_{\text{best}} \leftarrow \emptyset$ 
2:  $n_{\text{best}} \leftarrow 0$ 
3: for  $k = 1$  to  $K$  do
4:   Randomly sample a minimal set  $\mathcal{S}$  of 8 correspondences from  $\mathcal{C}$ 
5:   Estimate  $F$  from  $\mathcal{S}$  (normalized 8-point method with rank-2 enforcement)
6:    $M \leftarrow \emptyset$ 
7:   for each correspondence  $(\mathbf{x}, \mathbf{x}') \in \mathcal{C}$  do
8:     Compute Sampson distance  $d_S(\mathbf{x}, \mathbf{x}', F)$ 
9:     if  $d_S(\mathbf{x}, \mathbf{x}', F) < \epsilon$  then
10:      Add  $(\mathbf{x}, \mathbf{x}')$  to  $M$ 
11:     end if
12:   end for
13:   if  $|M| > n_{\text{best}}$  then
14:      $M_{\text{best}} \leftarrow M$ 
15:      $n_{\text{best}} \leftarrow |M|$ 
16:   end if
17: end for
18: Estimate  $F$  from  $M_{\text{best}}$  return  $F$ 

```

to the epipolar constraint $\mathbf{x}'^T F \mathbf{x} = 0$, which when expanded out with eq. (3) and eq. (4) gets us eq. (5).

$$\mathbf{x}_i = \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}, \quad \mathbf{x}'_i = \begin{bmatrix} u'_i \\ v'_i \\ 1 \end{bmatrix} \quad (3)$$

$$\mathbf{f} = [f_{11} \ f_{12} \ f_{13} \ f_{21} \ f_{22} \ f_{23} \ f_{31} \ f_{32} \ f_{33}]^T \quad (4)$$

$$[x_i x'_i \ x_i y'_i \ x_i \ y_i x'_i \ y_i \ x'_i \ y'_i \ 1] \mathbf{f} = 0 \quad (5)$$

This setup allows us to use SVD to solve the equation with our 8 correspondences. It is important to afterwards establish the Rank 2 Constraint (eq. (9)) to make sure that all of the epipolar lines intersect at a point. This crucial step ensures that the estimated camera pose (which uses the Fundamental Matrix), is accurate and that the error is not carried over to all of the remaining steps of the framework. The way to do

this is to make the smallest singular value $s_3 = 0$. Keeping all three singular values like in eq. (8) means there's no null space and no epipole.

$$A\mathbf{f} = 0, \quad A \in \mathbb{R}^{8 \times 9} \quad (6)$$

$$A = U\Sigma V^T, \quad \mathbf{f} = \underset{\|\mathbf{f}\|_2=1}{\operatorname{argmin}} \|A\mathbf{f}\|_2 \quad (7)$$

$$F = U_F \operatorname{diag}(\sigma_1, \sigma_2, \sigma_3) V_F^T, \quad \sigma_1 \geq \sigma_2 \geq \sigma_3 \quad (8)$$

$$F_{\text{rank2}} = U_F \operatorname{diag}(\sigma_1, \sigma_2, 0) V_F^T \quad (9)$$

Like stated in the previous section, the fundamental matrix is then taken through RANSAC to make sure that there are no outliers in the 8 chosen correspondences negatively affecting F .

C. Estimate Essential Matrix from Fundamental Matrix

The essential matrix E , which assumes that the cameras follow the pinhole model, takes advantage of the the computed F to find the relative camera poses between two images, eq. (10). Instead of relating two points like F , it relates two cameras with the eq. (11).

$$E = K^T F K \quad (10)$$

$$\mathbf{x}'^T E \mathbf{x} = 0 \quad (11)$$

D. Estimate Camera Pose from Essential Matrix

Once the essential matrix is calculated, algorithm algo-rithm 2 was used to extract the possible camera poses for the second image. Because the F and E deal with relative poses, the first image is set to the origin, and the pose for the camera of the second image is "built" off that. The pose consists of $P = K[R|t]$, so the goal of the algorithm is to get the translation candidates and the Rotation matrix.

Singular Value Decomposition is used on E , getting us two possible translations: $T = u_2$ or $T = u_3$ because the epipole will lie on the plane. The left nullspace is in the second image and $T^T \times E = 0$. With that information, we go for the calculation of the rotation matrix, which we aim go to acquire with eq. (12), and eq. (13)

Algorithm 2 Extracting Camera Poses from E

Require: Essential matrix $E \in \mathbb{R}^{3 \times 3}$ **Ensure:** Four possible camera poses $\{[R_i | t_i]\}_{i=1}^4$

1: Compute Singular Value Decomposition:

$$E = U\Sigma V^T$$

2: Define matrix

$$W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3: Compute two possible rotations:

$$R_1 = UWV^T, \quad R_2 = UW^TV^T$$

4: Extract translation direction (up to scale):

$$t = \mathbf{u}_3$$

where $\mathbf{u}_3$ is the third column of U

5: Define translation candidates:

$$t_1 = t, \quad t_2 = -t$$

6: Initialize pose set $\mathcal{P} \leftarrow \emptyset$ 7: **for** each $R \in \{R_1, R_2\}$ **do**8: **for** each $t \in \{t_1, t_2\}$ **do**9: **if** $\det(R) < 0$ **then**10: $R \leftarrow -R$ 11: $t \leftarrow -t$ 12: **end if**13: Append $[R | t]$ to $\mathcal{P}$ 14: **end for**15: **end for****return** $\mathcal{P}$

$$E = [t]_X R \quad (12)$$

$$T_X = u_1 \times u_2 = U \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} U^T \quad (13)$$

In the end we are left with two possible translation vector and two possible rotation matrices, which ends in 4 possible camera poses (one per combination of t and R , which have to be disambiguated. This is done with the Chirality Constraint, which looks at the points relate to each of the camera poses. ?? shows the world points and how they look different for each of the possible poses.

E. Triangulation Check for Chirality Condition

To disambiguate the poses, the Chirality Constraint is used, which looks at all of the world points in respect to each of the possible camera poses and then selects the one in which most points are in the positive Z direction. This is accomplished with triangulation. Before making any comparisons between world points and poses, we need to reconstruct said points from projections with a method called triangulation. This

uses two corresponding image projections from two different cameras and sees where their rays intersect, getting the world point X . We first accomplish this using linear triangulation and the following algorithm 3.

Algorithm 3 Pose Disambiguation with Linear Triangulation

Require: Candidate poses $\{[R_i | t_i]\}_{i=1}^4$, intrinsic matrix K , correspondences $\{(\mathbf{x}_j, \mathbf{x}'_j)\}_{j=1}^N$ **Ensure:** Correct camera pose $[R^* | t^*]$ and triangulated 3D points $\{\mathbf{X}_j^*\}$

1: Define reference pose:

$$P_1 = K[I | 0]$$

2: **for** each candidate pose $[R_i | t_i]$ **do**

3: Compute projection matrix:

$$P_2 = K[R_i | t_i]$$

4: Triangulate all correspondences:

$$\mathbf{X}_j^{(i)} = \text{LinearTriangulation}(P_1, P_2, \mathbf{x}_j, \mathbf{x}'_j)$$

5: Count points satisfying cheirality:

$$Z_{1j} > 0 \quad \text{and} \quad Z_{2j} > 0$$

where

$$Z_{2j} = \mathbf{r}_3^T \mathbf{X}_j^{(i)} + t_3$$

6: **end for**

7: Select pose with maximum number of valid points

return Best pose, corresponding triangulated points

1) *Linear Triangulation:* Linear triangulation estimates the position of a 3D world point given its projections in various images and the camera poses for them. Each 2D point x_i in image i defines a ray through the camera center, following the pinhole camera model eq. (14). These rays are supposed to intersect at the 3D point X , but due to noise they don't exactly. This is solved by using a linear system like in eq. (15), where the cross-product term enforces that the new point X is on the ray corresponding to both projected rays. The system $AX = 0$ is solved with SVD, which gets us X as seen in the last section To solve this, a linear system is constructed as shown in eq. (16). Figure 4 shows an example of triangulated points using this method for images 1 and 2. Figure 2 shows the reprojected points after linear triangulation for image 2.

$$\lambda \begin{bmatrix} x \\ 1 \end{bmatrix} = P \begin{bmatrix} X \\ 1 \end{bmatrix} \quad P = K[R | t] \quad (14)$$

$$A\mathbf{X} = \begin{bmatrix} \begin{bmatrix} x_1 \\ 1 \end{bmatrix} \times P_1 \\ \begin{bmatrix} x_2 \\ 1 \end{bmatrix} \times P_2 \\ \vdots \\ \begin{bmatrix} x_N \\ 1 \end{bmatrix} \times P_N \end{bmatrix} \begin{bmatrix} X \\ 1 \end{bmatrix} = 0 \quad (15)$$

Linear Reprojection of 3D points for image 2

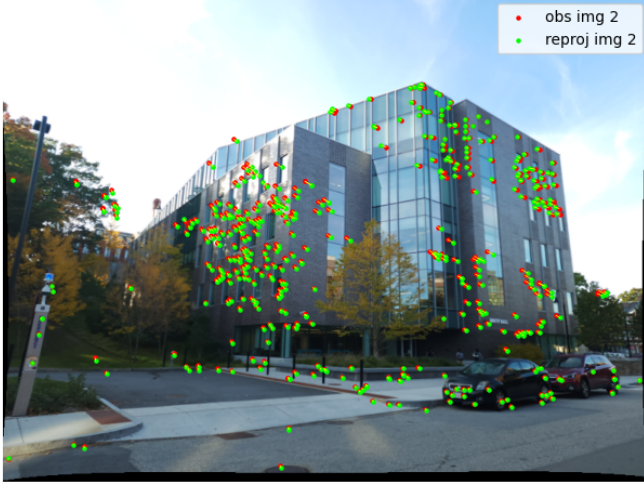


Fig. 2. Reprojected points after linear triangulation for image 2

Nonlinear Reprojection of 3D points for image 2

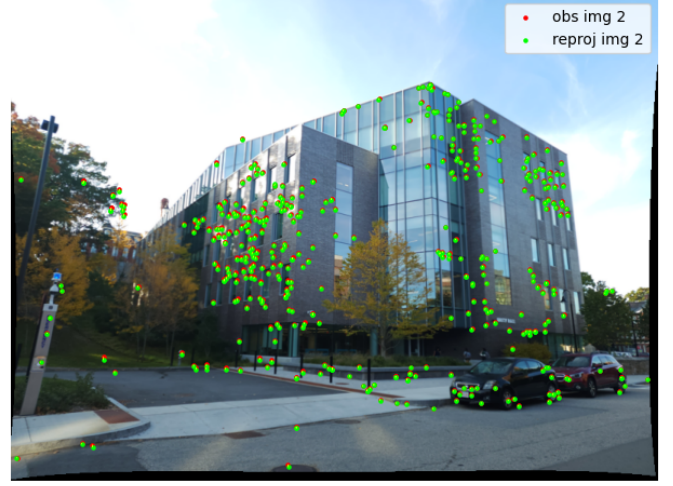


Fig. 3. Reprojected points after nonlinear triangulation for image 2

$$A = U\Sigma V^T \quad \mathbf{X} = \underset{\|\mathbf{X}\|=1}{\operatorname{argmin}} \|A\mathbf{X}\| \quad (16)$$

2) *Non-Linear Triangulation*: Non-linear triangulation does not replace its linear counterpart, instead, it builds from it and refines the 3D points it created by explicitly minimizing the reprojection error in image space. Each 3D point X is first projected into image coordinates using the camera's projection matrix as shown in eq. (17), and then converted to pixel coordinates by dehomogenization to produce $(\hat{u}, \hat{v})$. The reprojection error $r(X)$, defined in eq. (19), represents the difference between the projected point from X and the observed image point across all views. We used least squares optimization to get the optimal 3D point X^* as seen in eq. (20). This refinement makes the 3D reconstruction more accurate than the linear triangulation, especially when there's noise in the system and the camera poses estimated are not perfect.

An example of reprojected points after non-linear triangulation can be found in Figure 3. Note the decreased visibility of red dots (observed points) in the image compared to Figure 2, suggesting the success of our implementation.

$$P\mathbf{X} = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \end{bmatrix}, \quad P = K[R | t] \in \mathbb{R}^{3 \times 4} \quad \mathbf{X} = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (17)$$

$$\pi(P, \mathbf{X}) = P\mathbf{X} \frac{1}{\tilde{z}} = \begin{bmatrix} \tilde{x}/\tilde{z} \\ \tilde{y}/\tilde{z} \end{bmatrix} = \begin{bmatrix} \hat{u} \\ \hat{v} \end{bmatrix} \quad (18)$$

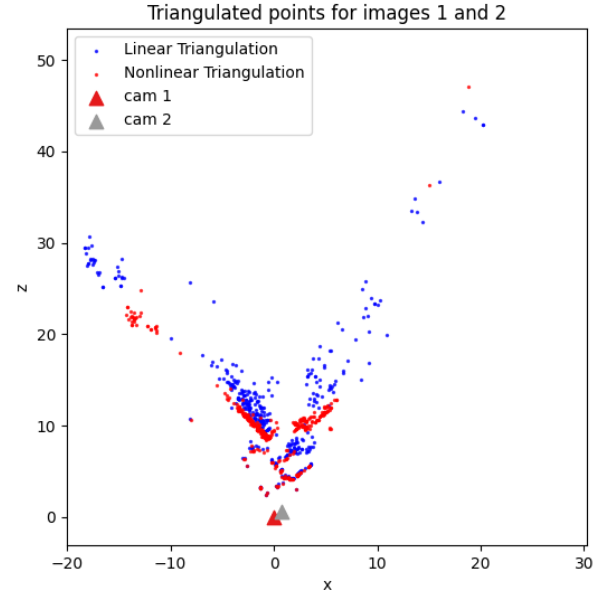


Fig. 4. Top-down view of linear and nonlinear triangulated points from the base pair (images 1-2)

$$\mathbf{r}(\mathbf{X}) = \begin{bmatrix} \mathbf{x}_1 - \pi(P_1, \mathbf{X}) \\ \mathbf{x}_2 - \pi(P_2, \mathbf{X}) \end{bmatrix} = \begin{bmatrix} u_1 - \hat{u}_1(\mathbf{X}) \\ v_1 - \hat{v}_1(\mathbf{X}) \\ u_2 - \hat{u}_2(\mathbf{X}) \\ v_2 - \hat{v}_2(\mathbf{X}) \end{bmatrix} \in \mathbb{R}^4 \quad (19)$$

$$\mathbf{X}^* = \underset{\mathbf{X} \in \mathbb{R}^3}{\operatorname{argmin}} \|\mathbf{r}(\mathbf{X})\|_2^2 \quad (20)$$

F. Perspective-n-Points

The previous steps have given us a set of good world points based on two images, which have been refined through non-linear triangulation, but continuing this method for more cameras is not viable. Repeating these previous steps with an already calculated image and the new view would be a less robust strategy, and the noise would accumulate, instead, Perspective-n-Points uses all of the previously triangulated world points to make sure the new estimate for the camera pose is more accurate.

1) *Linear Camera Pose Estimation (PnP)*: Linear PnP is in a RANSAC algorithm to estimate the pose of a camera given a set of 3D points and their corresponding 2D projections in the image. The estimated pose is calculated by making a linear system out of each correspondence and solving it with SVD, as seen in algorithm 4. The singular value decomposition minimizes the algebraic error and provides a projection matrix from which the rotation matrix and translation vector have to be corrected because the linear least square solve doesn't enforce orthogonality of R , as it is not geometric. For this process, six correspondences are required because the system needs to solve 12 degrees of freedom (9 from the rotation and 3 from the translation), and each correspondence brings two equations.

Algorithm 4 Linear PnP

Require: N correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}$ with $\mathbf{X}_i \in \mathbb{R}^3$, $\mathbf{x}_i = (u_i, v_i)^T$, intrinsics K

Ensure: Pose (R, t)

- 1: Form homogeneous points $\tilde{\mathbf{X}}_i = [X_i \ Y_i \ Z_i \ 1]^T$, $\tilde{\mathbf{x}}_i = [u_i \ v_i \ 1]^T$
 - 2: Build a linear system $\mathbf{A}\mathbf{p} = \mathbf{0}$ from the constraint $\tilde{\mathbf{x}}_i \times (P\tilde{\mathbf{X}}_i) = \mathbf{0}$, where $\mathbf{p} = \text{vec}(P)$ and $P \in \mathbb{R}^{3 \times 4}$
 - 3: Solve $\mathbf{A}\mathbf{p} = \mathbf{0}$ via SVD and reshape $\mathbf{p}$ into P
 - 4: Let $P = [P_{3 \times 3} \ \mathbf{p}_4]$ and compute $M = K^{-1}P_{3 \times 3}$
 - 5: Project M to the closest rotation by SVD: if $M = U\Sigma V^T$, set $R \leftarrow UV^T$
 - 6: Set scale $s \leftarrow \Sigma_{11}$ and translation $t \leftarrow \frac{1}{s}K^{-1}\mathbf{p}_4$
 - 7: **if** $\det(R) < 0$ **then**
 - 8: $R \leftarrow -R$, $t \leftarrow -t$
 - 9: **end if**
 - return** (R, t)
-

2) *PnP RANSAC*: Like fundamental matrix estimation (Section II-A), linear PnP was implemented inside of RANSAC. The error metric for finding inliers can be found in Equation (21).

$$\hat{\mathbf{x}}_i = \Pi(P\tilde{\mathbf{X}}_i), \quad e_i = \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2, \quad \text{inlier if } e_i < \epsilon. \quad (21)$$

An example of 3D points, reprojected back to the image plane, is found in Figure 6. Yellow lines in the image connect the observed (ground truth) points and their reprojected counterparts.

Linear PnP Reprojection of 3D points for image 4

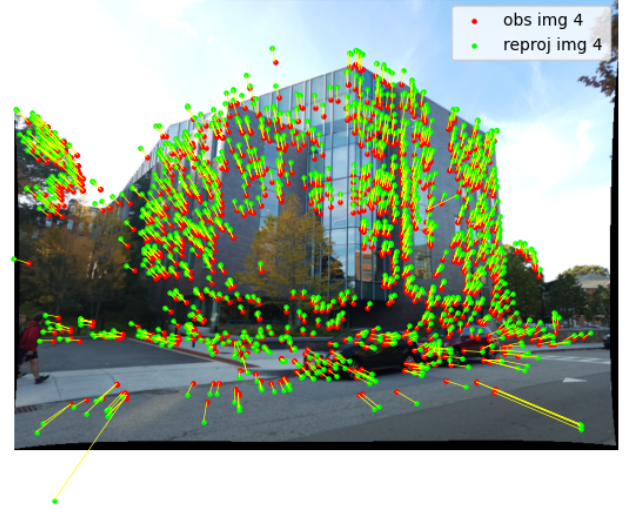


Fig. 5. Reprojected points after linear PnP for image 4

3) *Nonlinear PnP*: Like for triangulation, the linear Perspective-n-Point solution can be refined using nonlinear PnP, which minimizes the geometric reprojection error between measured 2D points and projected 3D points (eq. (25)). In this optimization the rotation of each camera pose is encoded as a quaternion $\mathbf{q}$ (eq. (22)), which provides a compact and more efficient representation for optimization. We used nonlinear least squares to optimize the full set of pose parameters $\boldsymbol{\theta} = [\mathbf{t}; \mathbf{q}]$ (eq. (26)–eq. (27)). The difference between this formulation and that for nonlinear triangulation is that the parameters for PnP describe the camera pose as opposed to the 3D point positions. The optimization returns a refined rotation R^* and translation $\mathbf{t}^*$ that minimize the reprojection error, which makes the camera poses more accurate.

$$R(\mathbf{q}) \in SO(3), \quad \mathbf{q} \in \mathbb{R}^4 \quad (22)$$

$$\boldsymbol{\theta} = \begin{bmatrix} \mathbf{t} \\ \mathbf{q} \end{bmatrix} \in \mathbb{R}^7, \quad \mathbf{t} \in \mathbb{R}^3 \quad (23)$$

$$\tilde{\mathbf{x}}_i = P(\boldsymbol{\theta})\tilde{\mathbf{X}}_i = \begin{bmatrix} \tilde{x}_i \\ \tilde{y}_i \\ \tilde{z}_i \end{bmatrix}, \quad \hat{\mathbf{x}}_i = \begin{bmatrix} \hat{u}_i \\ \hat{v}_i \end{bmatrix} = \begin{bmatrix} \tilde{x}_i/\tilde{z}_i \\ \tilde{y}_i/\tilde{z}_i \end{bmatrix} \quad (24)$$

$$\mathbf{r}(\boldsymbol{\theta}) = \begin{bmatrix} \mathbf{r}_1(\boldsymbol{\theta}) \\ \vdots \\ \mathbf{r}_N(\boldsymbol{\theta}) \end{bmatrix} = \begin{bmatrix} u_1 - \hat{u}_1 \\ v_1 - \hat{v}_1 \\ \vdots \\ u_N - \hat{u}_N \\ v_N - \hat{v}_N \end{bmatrix} \in \mathbb{R}^{2N} \quad (25)$$

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \|\mathbf{r}(\boldsymbol{\theta})\|_2^2 \quad (26)$$

Nonlinear PnP Reprojection of 3D points for image 4

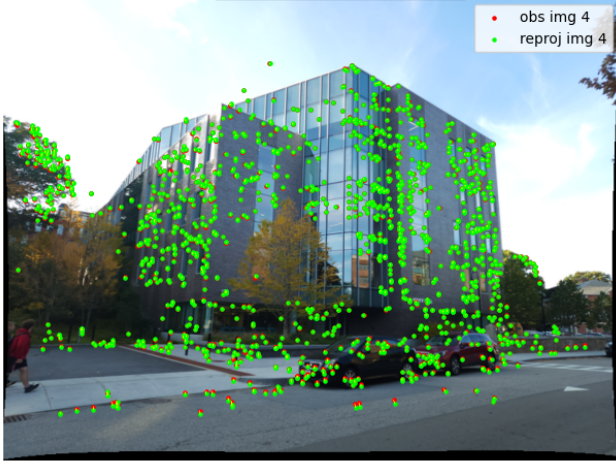


Fig. 6. Reprojected points after nonlinear PnP for image 4

$$\mathbf{q}^* = \frac{\boldsymbol{\theta}_{4:7}^*}{\|\boldsymbol{\theta}_{4:7}^*\|}, \quad R^* = R(\mathbf{q}^*), \quad \mathbf{t}^* = \boldsymbol{\theta}_{1:3}^* \quad (27)$$

After the camera poses have been refined, new points are added to the map by triangulating correspondences between the new camera and 2D points in the already included frames. PnP makes this addition of points more consistent with the existing map, making the SfM reconstruction more robust. The map after adding all 5 cameras with this method can be seen in Figure 7. Reprojected 3D points are visualized in Figure 6 (compare with the linear equivalent in Figure 5).

G. Bundle Adjustment

The final part of the SfM framework is to make a final refinement of all of the points and poses by passing everything into an optimization function. This minimization observes how variations in X or in the camera pose affect the geometric error. This step looks at the relationship between every camera pose and each world point X through a Visibility Matrix.

1) *Visibility Matrix*: The Visibility Matrix denotes the relationship between a point in the world and a camera pose by seeing if a point can be observed by each camera. This acts as a mask for the bundle adjustment to avoid wasting time on computations for points that are not visible to a camera, which could be a problem due to the massive size of the matrix.

2) *Bundle Adjustment*: To reduce the accumulated errors throughout the whole framework, we ran a final optimization of the world points and the camera poses. For the bundle adjustment, the camera rotations are represented with rotation vectors and packed along with translations and 3D points into a single parameter vector for optimization. The residuals are computed as difference between the observed 2D points and the projection of their 3D counterpart through each camera's projection matrix in the same way as it has been done for the nonlinear triangulation and PnP, (Equations (17)–(20) and (25)). The visibility matrix was used to make a sparsity pattern

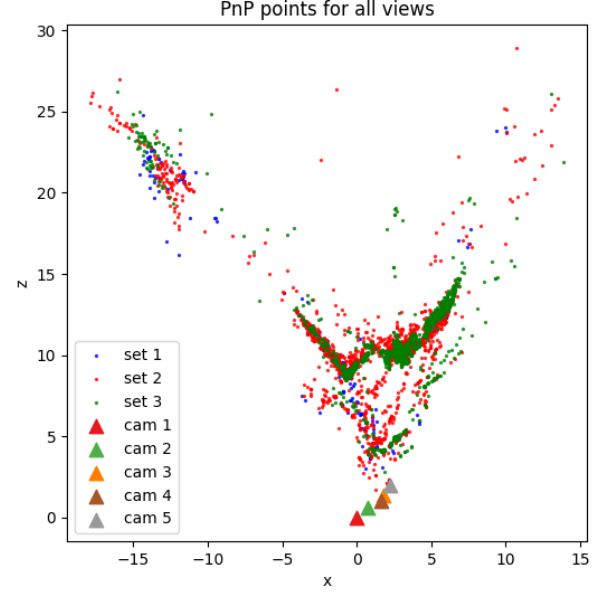


Fig. 7. World point estimates from nonlinear PnP (top-down view)

TABLE I
MEAN REPROJECTION ERROR ACROSS ALL RELEVANT VIEWS AFTER EACH STEP OF THE PIPELINE

	Mean Reprojection Error (px)
Linear Triangulation	5.1735
Nonlinear Triangulation	2.3370
Linear PnP	17.908
Nonlinear PnP	1.785
Bundle Adjustment	0.0609

of the Jacobian so that the derivatives are only computed for the camera-point pairs that are connected (a camera-pair is connected if the world point is visible by the camera in question). Due to the large number of points in the map, this greatly improves computational efficiency.

Finally, a least squares solver simultaneously refines the camera poses and 3D points to improve the accuracy of the SfM reconstruction and minimize the errors due to compounding noise in every step. A final map of all the points for the 5 images before and after the Bundle Adjustment can be seen in fig. 9. In this case, the improvement is not immensely visible, but that is not a concern for us, since the PnP results were already very accurate, and five images might not be enough to showcase the accumulating error of SfM. You can see the corner of the building very clearly, and the improvement throughout all of the steps is also evident as showcased in the reprojection errors presented in Table I. Refer to Figure 8 to see reprojected points for image 5.

H. Putting the pipeline together

Details regarding how the previous sections all come together can be clearly seen in Algorithm 5.



Fig. 8. Reprojected bundle adjustment points for image 5

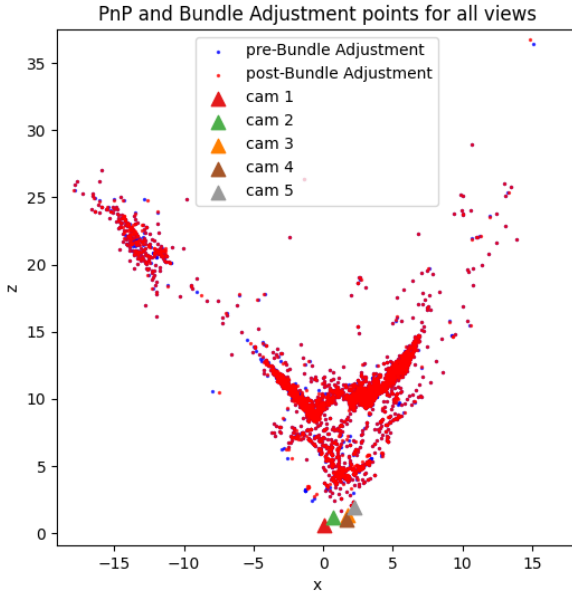


Fig. 9. World point estimates before and after bundle adjustment (top-down view)

Algorithm 5 High-level SfM Pipeline

Require: Image IDs (i_1, i_2) , optional additional image IDs $\{i_k\}$, intrinsics K

Ensure: Refined camera poses $\{[R_j|t_j]\}$ and 3D points $\{\mathbf{X}_m\}$

- 1: Load 2D correspondences between i_1 and i_2
- 2: Estimate F via RANSAC using $\mathbf{x}^T F \mathbf{x} = 0$
- 3: Compute essential matrix:

$$E = K^T F K$$

- 4: Extract four candidate poses $\{[R|t]\}$ from E
- 5: Fix first camera as $[I|0]$
- 6: Disambiguate second camera using cheirality condition:

$$Z_1 > 0, \quad Z_2 > 0$$

- 7: Linearly triangulate base 3D points
- 8: Refine base 3D points via nonlinear triangulation
- 9: **for** each additional image i_k **do**
- 10: Build 2D–3D correspondences with existing 3D map
- 11: Estimate pose using PnP-RANSAC
- 12: Refine pose using nonlinear PnP
- 13: Triangulate additional 3D points with existing views
- 14: **end for**
- 15: Form all projection matrices:

$$P_j = K[R_j|t_j]$$

- 16: Assemble global 3D point set $\{\mathbf{X}_m\}$
- 17: Construct visibility matrix V where

$$V_{jm} = \begin{cases} 1 & \text{if camera } j \text{ observes point } m \\ 0 & \text{otherwise} \end{cases}$$

- 18: Perform bundle adjustment:

$$\min_{\{R_j, t_j\}, \{\mathbf{X}_m\}} \sum_{j,m} V_{jm} \left\| \mathbf{x}_{jm} - \Pi \left(K[R_j|t_j] \tilde{\mathbf{X}}_m \right) \right\|_2^2$$

return Optimized poses and 3D points

III. PHASE 2: DEEP LEARNING APPROACH

In Phase 2 of this project, we present an implementation of Neural Radiance Fields (NeRF) [3] to synthesize novel views of complex scenes by using optimization and a deep network that takes an input of a position and viewing direction and outputs a volume density and an emitted radiance at that location. The differentiable nature of the volume rendering means that this can be achieved by computing the loss with a comparison between generated images and a set of ground truth images at known pose locations.

A. Dataset and Scene Representation

Datasets for NeRF consist of a set of images and known camera poses. For this project, we sought to train novel view synthesis systems for the Lego and ship datasets, as seen in Figures 11 and 12.

For the purpose of this project, we created a data loader to prepare the set of images to inputs for the model. The

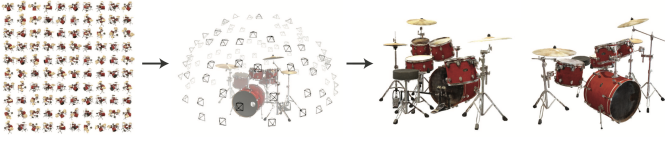


Fig. 10. With an input of images at known camera poses, an optimization of the NeRF allows the model to render novel views.



Fig. 11. Sample view of Lego dataset, from [3].

model will take a 3D location $x = (x, y, z)$ and a direction (θ, ϕ) , with which it emits a color $c = (r, g, b)$ and a volume density σ , Figure 13. This means that set of images had to be converted into a set of rays, (a position and direction), to generate the batches. These rays are generated by unprojecting the pixel coordinates through the inverse of the intrinsic matrix K , which then gets transformed to the world space with the rotation R from the known camera pose. This along with the translation t , which is the shared origin for all the pixels in the image, gives us the ray $r(t) = o + td$, as per Section 4 of [3].

During the course of this project, we encountered multiple issues with the training time, which, if too high, meant that we



Fig. 12. Sample view of ship dataset, from [3].

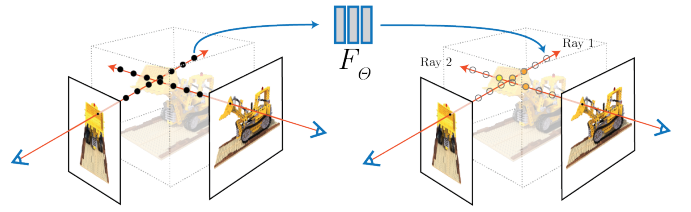


Fig. 13. Through volume rendering, NeRF synthesizes novel views by querying points along rays from the camera center (i.e. positions x, y, z and viewing angles θ, ϕ).

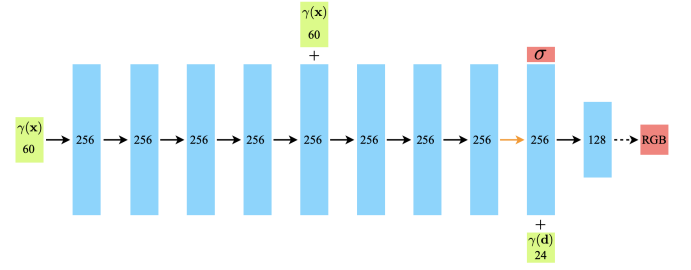


Fig. 14. Network architecture employed for Phase 2. Figure from [3].

couldn't test the model as often and we should, having to wait to see the results difference of a small change. One way that this was tackled was with the loading of the data; originally, we had the generation of rays at every batch creation, which was giving us training times of over 24 hours. We fixed this by precomputing all of the rays at the initialization so that `get_batch()` simply served as a lookup and we didn't have to perform rotation matrix multiplications over the course of the training. This was among the most important changes we made that permitted us to train the model in around 4 hours for 100k iterations.

B. Network Architecture and Pipeline

In the 5D Input of the network, separated into the 3D position and 2D viewing direction, the two components are not fed into the network together. The position goes first, and goes through 8 fully-connected layers with 256 channels each, ReLU activations, and a skip connection at the fifth layer [3]. After the 8 layers, the volume density and a 256-dimensional feature vector are outputted, which get concatenated to the ray direction for an additional fully connected layer with 128 channels that outputs the emitted RGB radiance at that 5D input, (at position x as seen by ray with direction d), fig. 14. For the final layer, to get the color output we followed [3] and used a sigmoid activation to constrain the color to $[0, 1]$, which wouldn't have been the case with ReLU.

The network receives a batch of rays, each defined by an origin o , and direction d . Each of these rays represents a pixel whose color we want to predict, which is why the pipeline operates on each ray. The overall pipeline leverages two instances of the network, a coarse and a fine network,

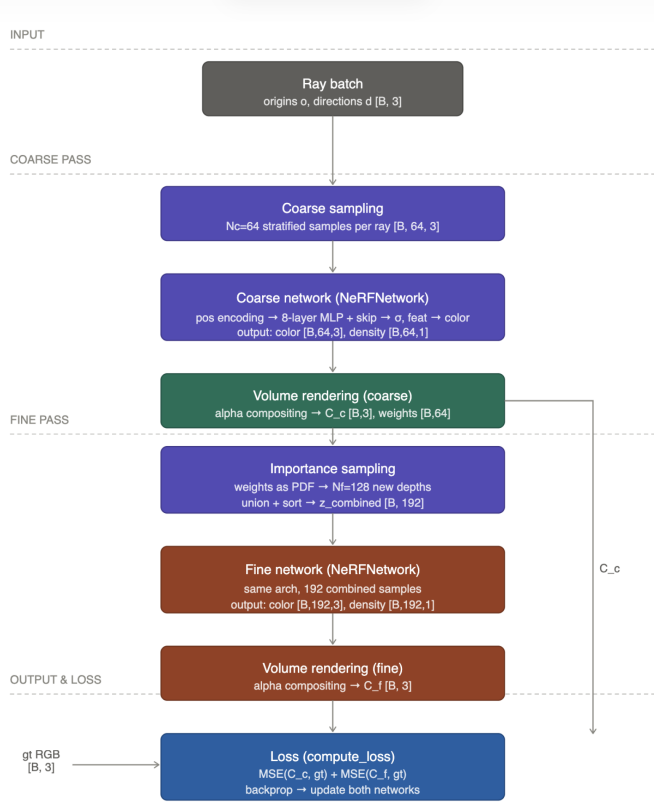


Fig. 15. Pipeline for the NeRF, as showcased in [3] and implemented for this second phase of the project.

which will be explained to more detailed in Section III-C. We begin by doing a coarse sampling, which uniformly partitions the ray between a near distance of 2 and a far of 6. We chose to sample each ray into $N_C = 64$ positions, which are then passed along with the the direction into the coarse network. The positions are then positionally encoded and run through the MLP after which they are joined with the direction through the second stage of the network to output a color and a density.

The color and density are passed into the Volume Rendering (Section III-D1), which outputs the 64 per-sample weights that represent how much each of the sampled points contribute to the final color. The weights represent what part of the ray is most interesting to further investigate with the fine network, samples for which are acquired with the weights as a Probability Density Function (PDF), to draw $N_f = 128$ samples at different depths with an emphasis on the regions where the coarse network identified a high weight. Finally, these fine depths and the original uniform depths are combined and passed through the second instance of the network, the fine network, to get the color and density. These are passed one last time through the volume rendering to get the final predicted color for that ray, Figure 15. This final fine color (C_f) is used along with the coarse color acquired from the first volume rendering pass (C_c) to calculate the loss that is used to optimize the model. The reason why there are

two different instances of this network is so that they are optimized differently and the coarse network can specialize in learning a rough estimate of densities and colors along the ray from uniform samples while the fine network uses non-uniform samples and specializes differently. The separation in instances prevents the gradients from coarse and fine samples from interfering.

C. Hierarchical Sampling

Due to size of these networks, uniform or stratified sampling along every ray becomes very computationally expensive, as most rays pass through empty space, which doesn't contribute to the final color, and the level of detailed needed to compute the intricate surfaces of the objects being modeled would require such a high number of samples that training with such a density on every ray would not be feasible. The solution to this is to allocate more samples only to the regions of interest, where there the surface will have a large impact on the final color, which is accomplished with hierarchical sampling.

The idea is that the model first samples a small number of points uniformly per ray (in this case $N_C = 64$), and from there, get the weights through volume rendering (Section III-D1), which determines how much each of the samples contributes to the final color. The weights are used as a PDF in the `Importance()` function, explained to more detail in Section III-D2. A Cumulative Distribution Function (CDF) is obtained from the PDF, which is used to draw $N_f = 128$ samples with linear interpolation within designated bins. These new points are more closely aligned with where mass actually exists. These are combined with the coarse samples to pass through the model in order to get a full view of the object, without losing details in the dense regions and still testing where the empty space lays.

D. Volume Rendering and Importance Sampling

1) *Volume Rendering*: The network outputs color and density at each discrete sampling point of a ray, but that cannot be compared against a ground truth image, which is why we need to get a single RGB value per pixel. With volume rendering, we can determine what color the camera actually sees as a ray passes through the surfaces we are trying to model. This has to be done all while being differentiable to allow backpropagation of the rendering loss into the network weights.

The expected color of a ray is described by the paper as $C(\mathbf{r})$ as seen in eq. (28), where the transmittance $T(t)$ is shown in eq. (29). The transmittance is the measure of how much of the light passes through a medium and it is what makes occlusion work correctly. A surface being intersected by the ray cannot contribute to the final color if there is another surface earlier in the ray that is opaque. This permits the network to properly predict the shape of an object based on what colors are intercepted first.

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (28)$$

$$T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right) \quad (29)$$

The volume rendering function used in the network uses a discrete approximation of the previously-stated equations for the sampled points based on keeping track of the deltas between sample points. These distances between samples, are also multiplied at some point by `torch.norm(direction)`, which scales the optical thickness so that a denser sampling doesn't inflate opacity. The deltas are used for calculating the alpha value (the opacity of a sample), eq. (30), which is later used along with the transmittance to get the weights at each of the samples, eq. (31).

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i) \quad (30)$$

$$w_i = T_i \cdot \alpha_i = T_i (1 - \exp(-\sigma_i \delta_i)) \quad (31)$$

To get the final RGB values, the weights per sample eq. (31) get multiplied by the color originally outputted by the coarse or fine network. The weights have a maximum value of 1 and are the key to portray what the scene is actually like. The closer the weight of a sample is to 1, the more likely that the ray hit something at that sample. This makes them very valuable as a PDF to select the fine samples within the Importance Sampling section.

2) *Importance Sampling*: The importance function takes in the weights from the coarse pass for each of the coarse samples along the ray. These denote where along the ray the object is likely to be. The first step is to turn them into a proper Probability Density Function by normalizing them with so that they can add up to 1, eq. (32). One important aspect that we have in this normalization is the small epsilon ϵ , which prevents division from zero but also ensures that every bin has a small non-zero probability. Having epsilon in the numerator was important to still sample empty regions, making the network more robust when the coarse network is in its early stages.

$$\hat{w}_i = \frac{w_i + \epsilon}{\sum_{j=1}^{N_c} (w_j + \epsilon)} \quad (32)$$

From the PDF, the CDF creates an increasing staircase function between 0 and 1 where the larger steps belong to the regions with a high probability of encountering a surface along the ray, eq. (33). From this, inverse transform sampling can draw N_f samples into a uniform distribution $u \sim \mathcal{U}[0, 1]$, and the inverse $t^* = \text{CDF}^{-1}(u)$, where t^* is the new sample. By detecting the bin / step of the inverse of CDF where u "falls", you can get the bin edges and interpolate the depth. The discrete nature of the CDF means that the inversion cannot be used and that it is replaced by binary search in the form of `torch.searchsorted(cdf, u, right=True)`. The returned index you can get the edges of the bin (eq. (34)),

from which linear interpolation gets you the depth of the new sample, eq. (35).

$$\text{CDF} = \left[0, \hat{w}_1, \hat{w}_1 + \hat{w}_2, \dots, \sum_{i=1}^{N_c} \hat{w}_i\right] \in \mathbb{R}^{N_c+1} \quad (33)$$

$$k_{\text{above}} = \text{clamp}(k, 0, N_c - 1) \quad k_{\text{below}} = \text{clamp}(k - 1, 0, N_c - 1) \quad (34)$$

$$t^* = t_{k_{\text{below}}} + \frac{u - \text{CDF}_{k_{\text{below}}}}{\text{CDF}_{k_{\text{above}}} - \text{CDF}_{k_{\text{below}}}} \cdot (t_{k_{\text{above}}} - t_{k_{\text{below}}}) \quad (35)$$

One final challenge about this implementation was matching the weights (which are defined between sample points) and the coarse samples. The CDF had to be built over the midpoints between adjacent coarse samples $z_{\text{mid},i} = \frac{z_i + z_{i+1}}{2}$ rather than the samples themselves, since each weight w_i describes what happened in the interval between sample i and $i + 1$, not at either endpoint. This also requires trimming the first and last weights, as the outermost intervals lack a second neighbor and are therefore not well-defined in the same way.

E. Loss Function

The loss function for this network is quite simple. We used the sum of two mean squared errors to calculate the total loss, one gotten from the comparison between the coarse rendered color against the ground truth and the other using the fine rendered color, eq. (36). R is the set of rays in the batch, $C(r)$ is the ground truth RGB and C_c and C_f are the coarse and fine rendered colors.

$$\mathcal{L} = \sum_{r \in R} \left[\left\| \hat{C}_c(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 \right] \quad (36)$$

We used mean squared error because it penalizes large deviations quadratically, meaning that the network is encouraged to get the overall geometry right before going into the finer detail. Another interesting point is the use of the coarse loss. Although it seems like the coarse network simply produces the weights for the fine sampling and that it is not crucial to properly train, this is not the case. If the coarse component of the loss wasn't included in the total loss calculation, the coarse network would not learn the weight distribution properly, and the arbitrary outputs would make the importance sampling no better than uniform sampling. This negates the efforts of the hierarchical architecture, which is why we use both components in the loss calculation.

F. Training Details

The training loop for our network is quite standard. We optimize the coarse and fine networks jointly; at each iteration a batch of rays is sampled and passed through the full hierarchical pipeline, and the combined MSE loss is backpropagated through both networks at the same time. Following the paper, [3], we used a batch size of 4096 rays drawn uniformly from

the full set of training images. Each ray comes from a different viewpoint, which provides a higher diversity than rendering an image at a time. We used the Adam optimizer with an initial learning rate of $5 * 10^{-4}$, which decays exponentially throughout the full training down to $5 * 10^{-5}$. Doing 100,000 iterations with 4096 rays each means that we see a total of about $4 * 10^8$ ray samples out of an estimate of 64 million available rays (the Lego set has 100 images at 800x800 pixels), this means that each ray is sampled about 6 times.

To speed up the training, we used PyTorch’s GradScaler and autocast for mixed precision training on CUDA. Implementing this change allowed us to really minimize our memory usage and increase our speed when training on Turing. In addition, we also implemented a downscaling CLI argument `--down` that scales the images by a set factor with `cv2.INTER_AREA`, this also improves the iteration speed by reducing the number of pixels per image making data loading and rendering faster, at the cost of a lower resolution at finer details.

1) *Positional Encoding*: A problem with MLP’s, explained in section 5.1 of the NeRF paper [3], is that they are biased towards learning low-frequency functions, which means that high-frequency features, like a repetitive pattern in the Lego bulldozer’s tracks, are smoothed out and not captured correctly. The sharp edges would not be properly caught, which is why positional encoding is implemented.

Positional encoding ensures the network can represent the high spatial frequencies needed for a proper render by mapping the coordinates into a higher-dimensional space with sinusoids at exponentially increasing frequencies, eq. (37). We used $L_{pos} = 10$ and $L_{dir} = 4$ for the different encoding parameters for the raw coordinates by following the reference paper [3]. L controls the highest frequency encoded; position has a finer frequency than direction because view-dependent effects vary much more smoothly and a higher L_{dir} would be wasteful.

$$\begin{aligned} \gamma(p) = & (p, \sin(2^0 \pi p), \cos(2^0 \pi p), \\ & \sin(2^1 \pi p), \cos(2^1 \pi p), \\ & \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p)) \end{aligned} \quad (37)$$

As can be seen in Figures 16 and 19, using positional encoding yielded marginally better results than abstaining from it, which confirms the speculations previously stated. Performing this operation on the raw coordinates is a better solution than complicating the raw-coordinate MLP with more layers, as that doesn’t guarantee a better result, as seen by the figure. It’s not a matter of converging or not, but whether the network can represent the functions needed to properly render the scene. Both examples in fig. 16 converged, but not using the positional encoding meant that the model was literally unable to represent the complexity needed.

G. Results

Our network’s performance was evaluated with two standard metrics for image quality and reconstruction: PSNR (Peak Signal-to-Noise Ratio) and SSIM (Structural Similarity Index



Fig. 16. A view of the Lego dataset synthesized by a model with no positional encoding. Compared to the ground truth (left), this version had a PSNR of 24.1254 dB. Without positional encoding, the render is much smoother and lacks details compared to the ground truth and to the result from the properly trained network seen in section III-G.

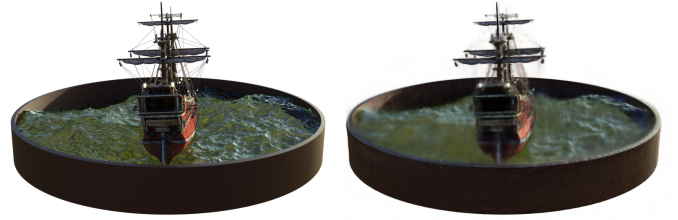


Fig. 17. Sample of a synthesized view of the ship dataset on the right. Ground truth on the left.

Measure). The Lego test dataset, the average PSNR was 26.8989 dB and the average SSIM was 0.8991. For the ship test dataset, the average PSNR was 25.6204 dB and the average SSIM was 0.8158. These values fall below the results reported in the reference paper [3]: 32.54 dB and 28.13 dB for the Lego and the ship datasets respectively, which is expected given the constraints that we established. The reference implementation was trained for up to 300k iterations on the full resolution for the images, while we had to downscale the images and ran much fewer iterations by a factor of 3. The gap between the Lego and the ship’s performance is also expected given the greater difficulty of the ship scene, which has more intricate details and higher contrasts that amplify any residual geometric error in PSNR and SSIM terms.

The visual results for both datasets in Figures 17 to 20 show that the model has successfully learned the overall geometry and color of both scenes, proving the competence of our network. Degradation is somewhat visible at the fine geometric boundaries of both the Lego and the ship in Figure 18, which can be attributed to the limited number of iterations and downsampling that we had to implement in order to train the network under the time constraint of this project.

The biggest issue we encountered during development was a misalignment between images and their corresponding camera



Fig. 18. Some samples of the novel synthesized views of the Lego and the ship datasets.

poses, which resulted in renders as seen in Figure 21. In the initial dataset implementation, images and poses were loaded through two separate passes; images were loaded by iterating over the directory using `sorted()`, while poses were read sequentially from the JSON file. We assumed that the alphabetical order from the `sorted` function would correspond with the alignment in the JSON file, but this was not the case. The result was that rays were being cast from a pose that did not correspond to the image it was paired with, making it impossible for the network to learn something relevant. It was asking the network to explain the color of a pixel in a photograph given the camera geometry of a different viewpoint. The fix was to unify the loading into a single pass over the JSON frames in `_load_images_and_poses()`, reading each image directly from the file path specified in its corresponding frame entry, which guarantees that image and pose are always in correspondence regardless of directory ordering.

An additional issue we had was poor handling of the images' alpha channel. Following our first attempt at training the model for the ship dataset, nothing was rendered due to our discarding of the alpha channel in the data loader. We would show a visualization for this, but since nothing was rendered, there is nothing to show for this issue. One can safely assume that the PSNR for that training instance would be extremely poor.

IV. CONCLUSION

A. Phase 1

This phase of this project saw a completion of 3D reconstruction from a series of 2D images with the use of a classical method. As opposed to deep learning, which abstracts a large amount of the math behind 2D to 3D modeling, we worked through every step of the Structure from Motion framework, preventing the build up of noise through multiple layers of optimization and a final adjustment to all of the camera poses and world points to minimize the geometric error between the

projected world points onto each of the image frames and the observed 2D points on the images themselves.

B. Phase 2

As seen in our evaluations on the test sets (Figures 17 and 19 to 21) and renderings of novel views (see Figure 18), our full implementation of NeRF was fairly successful. By following the architecture from Mildenhall et al, [3], we were able to reproduce novel view synthesis from a sparse set of images without explicit 3D supervision. Our rendered outputs demonstrated that the network has learned a geometrically accurate scene representation. While very successful in our implementation, we could improve our network in many different paths, such as improving speed [4] or generalization across scenes [5], which can elevate the model and increase its abilities.

V. CONTRIBUTION STATEMENT

Pau Alcolea: Function Development, Documentation and Writing, Network development **Ben Orr:** Integration, Figure Visualization, Function Development, Documentation and Writing, Rendering

REFERENCES

- [1] T. M. Inc., "Matlab version: 9.13.0 (r2022b)," Natick, Massachusetts, United States, 2022. [Online]. Available: <https://www.mathworks.com>
- [2] P. D. Sampson, "Fitting conic sections to "very scattered" data: An iterative refinement of the bookstein algorithm," *Computer Graphics and Image Processing*, vol. 18, no. 1, pp. 97–108, 1982. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0146664X82901010>
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," in *ECCV*, 2020.
- [4] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1–15, Jul. 2022. [Online]. Available: <http://dx.doi.org/10.1145/3528223.3530127>
- [5] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, "pixelnerf: Neural radiance fields from one or few images," 2021. [Online]. Available: <https://arxiv.org/abs/2012.02190>

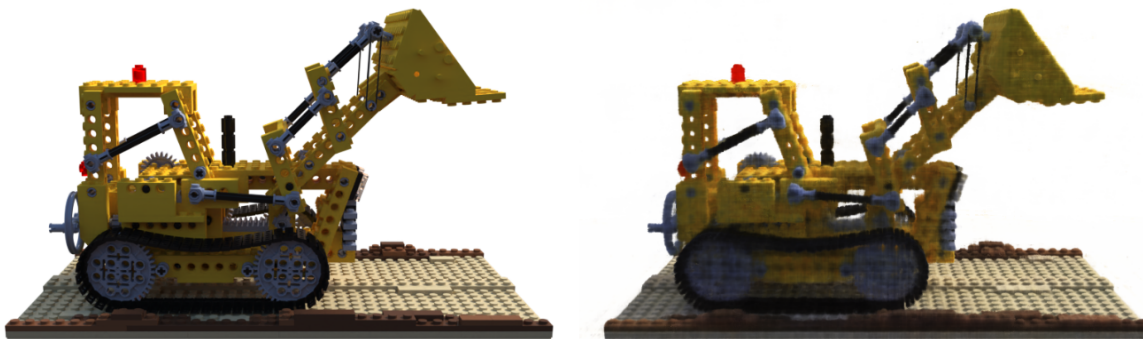


Fig. 19. A synthesized view of the Lego dataset on the right. The ground truth image on the left. This result is from our final model with positional encoding.

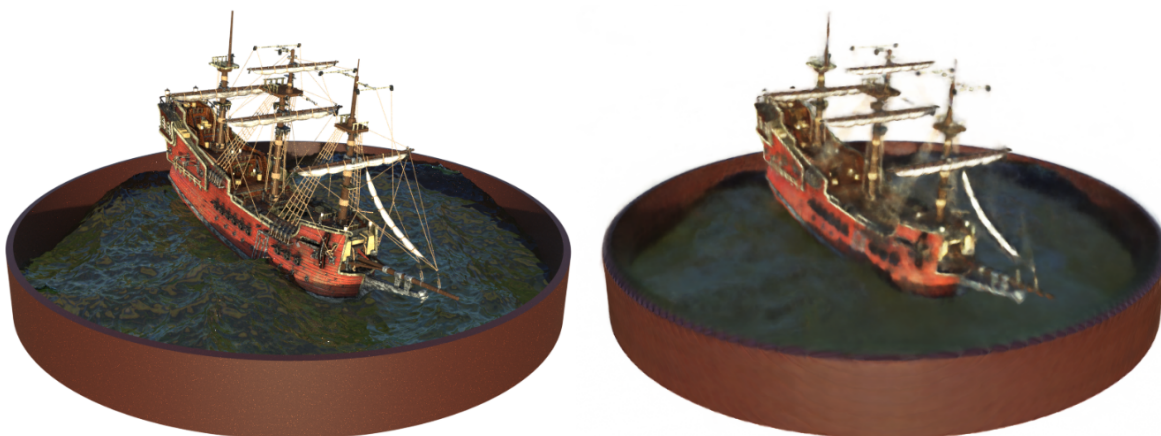


Fig. 20. A synthesized view of the ship dataset on the right. The ground truth image on the left.

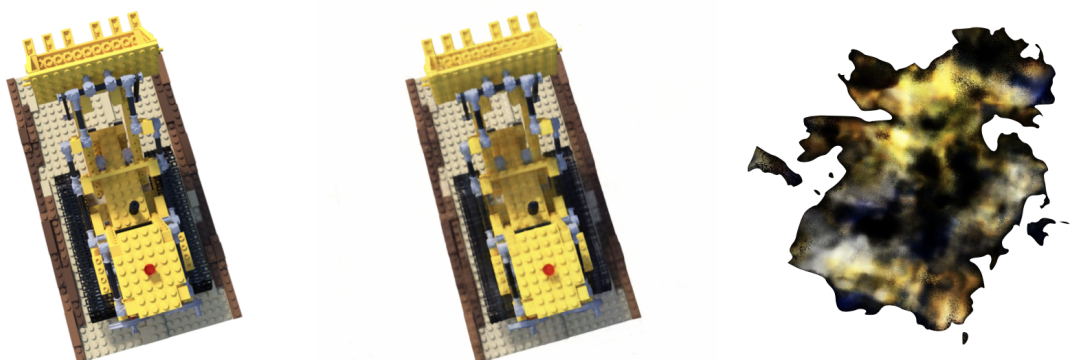


Fig. 21. An example of low quality rendering from the Lego dataset (right), with the corresponding ground truth on the left and our model's high quality rendering in the center. The issue shown on the right was the result of misalignment in images and camera poses.